

Admin Panel Audit

Thirty-six routed screens across eight modules, audited for function, visual system, UI/UX, architecture and backend schema. The build is green and the auth model holds. **Two findings are urgent:** the site-settings table hands every API key it stores to anonymous visitors, and the layering gate that CLAUDE.md relies on has been silently inert.

BRANCH	HEAD	DATE	LIVE SCHEMA
feature/dam-v3-milestone-planning	40ebb6d1	2026-09-11	64 tables · project iuuivmwq...
FINDINGS			
2 P0 · 8 P1 · 9 P2 · 7 P3			

01 – DOES IT WORK

Quality gates, run against this working tree

Every command below was executed. Two of the six fail; `npm run lint` is a blocking CI step, so **CI is red on this branch right now**.

npm run typecheck	npm run lint	npm run build	npm run arch:check
PASS	FAIL	PASS	FAIL
Root + web, clean	18 errors, 90 warnings	All bundle budgets met	3 reported · 32 real
arch:supabase-auth	arch:no-console		
PASS	PASS		
Proxy URL + no client session	No console.log in src/		

How far the live verification got

I cannot authenticate as the admin, so no screen behind the login was exercised. But `/admin/auth` renders inside `.admin-theme` without a session, which was enough to confirm findings **P1·6** and **P1·7** against a running dev server with real computed styles — both are now marked *verified in browser* below rather than inferred. Route guards were confirmed fail-closed the same way: an unauthenticated request for `/admin/system/settings` (super-admin only) lands on `/admin/auth`.

Everything else is from source, the production build output, and read-only queries against the live Supabase project. Still open: per-role RLS *write* policies (they need SQL access, not REST), deployment state of the unguarded edge functions, and the behaviour of the 30 authenticated screens. `e2e/admin-audit-verification.spec.ts` in the repo closes that last gap — it runs off the Playwright session file, so it needs the password once at setup and never handles it again.

02 - BLOCKING

Fix before the next deploy

P0 · 1

Every API key in `site_settings` is readable by anonymous visitors

`site_settings` carries `resend_api_key`, `supabase_api_key`, `vercel_api_key` and `posthog_api_key` as ordinary columns. `useSiteSettings` does `select("*")` and runs on every public page load, and RLS lets the public anon key read the whole row. So the studio's third-party credentials, `admin_email`, `security_config`, `rbac_permissions`, `telegram_chat_ids` and `report_recipients` are delivered to any visitor's browser and to anyone holding the anon key — which ships in the client bundle.

```
GET /rest/v1/site_settings?select=*&limit=1  apikey: <VITE_SUPABASE_ANON>  
→ HTTP 200, 1 row, 43 columns returned
```

```
AdminSettings.tsx:164-168  payload.resend_api_key    = integrationApiKey  
                           payload.supabase_api_key = integrationApiKey  
                           payload.vercel_api_key   = integrationApiKey  
useSiteSettings.ts:101    .from("site_settings").select("*")
```

Contrast this with the tables that *are* protected: `leads`, `profiles`, `user_roles`, `audit_logs`, `admin_sessions`, `quiz_results` and `raw_payload` all return zero rows to the anon key. The PII model is right; the settings table is the hole in it.

Fix: rotate the Resend, Vercel and PostHog keys, then move secrets out of the table into edge-function secrets (as `imagekit-upload` already does with the ImageKit private key). Split the row: a public view exposing only the branding/SEO/contact columns for `useSiteSettings`, with the sensitive columns restricted to an authenticated admin policy and read by an explicit column list, never `select("*")`.

VERIFIED LIVE

SECURITY

SECRET EXPOSURE

P0 • 2

The layering gate has never actually run

`.dependency-cruiser.cjs` declares nine `error`-severity rules, including `no-supabase-in-components`. It sets no `tsConfig` option, so `dependency-cruiser` cannot resolve the `@/` path alias — and virtually every import in this codebase uses it. Unresolvable targets never match a `to.path` regex, so the rules silently pass. The only three violations it has ever reported are the ones reachable through *relative* imports in test files.

```
npm run arch:check
  → 3 errors · 1219 modules cruised      (414 of those are phantom unre

npx depcruise src --config .dependency-cruiser.cjs --ts-config tsconfig.j
  → 32 errors · 805 modules cruised

29 × no-supabase-in-components      (18 under components/admin, incl. Asse
                                   CollectionDetailView, LeadDetailSheet
                                   UserFormSheet, plus AuthProvider and
```

CLAUDE.md states the layering is "enforced by tooling, not convention." It is currently enforced by neither. This is also why 27 components drifted into calling Supabase directly without anyone being told.

Fix: add `tsConfig: { fileName: "tsconfig.json" }` to the `options` block. Expect 29 new errors immediately — baseline them (the rule as written also catches `AuthProvider`, which legitimately owns the client, so it needs a `pathNot` exemption) and burn them down rather than reverting the fix. `arch:check` is `continue-on-error` in CI, so turning it on won't break the pipeline while you work through it.

VERIFIED

ARCHITECTURE

TOOLING

03 — HIGH

Broken function and broken surface

Screens that look operational but aren't, and two systemic styling faults that affect roughly four hundred class usages.

P1 • 3

Estimator pricing configuration is non-functional end to end

The `estimate_rates` table does not exist in the live database. Three consequences, all silent:

- **Load fails.** `EstimatorRegistry` tolerates only `PGRST116` (no rows); a missing table returns `PGRST205`, so the query throws.

`AdminEstimatorConfig` never reads `registry`'s error state, so the screen renders defaults as though they came from the database.

- **Save fails.** With `rateData?.id` undefined, `saveRates` always takes the `insert` branch, which 404s.
- **The public estimator ignores admin entirely.** `useEstimateEngine` catches the error and falls back to `DEFAULT_PRICING_CONFIG`. Client-facing quotes come from a hardcoded constant, not from anything an admin can change.

```
GET /rest/v1/estimate_rates?select=*&limit=1
→ HTTP 404 PGRST205 "Could not find the table 'public.estimate_rates'"

lib/registry/EstimatorRegistry.ts:20    if (error && error.code !== 'PGRS
useEstimateEngine.ts:31-41              catch → setPricingConfig(DEFAULT_
```

Related: only 5 of the 25 declared `FlowConfigKey`s are persisted in `estimator_flow_config` (`alcs_rules`, `discovery_archetypes`, `execution_tiers`, `property_types`, `result_templates`, all last written June 2026). The other twenty editors read hardcoded defaults, so their write paths have never been exercised.

VERIFIED LIVE

ESTIMATOR

PUBLIC IMPACT

P1 · 4

Roles page reports fabricated zeros, and hides the `editor` role

`AdminUserAccessRoles` counts users from `admin_users` — a table that does not exist. The query throws, nothing handles `isError`, and `roleCounts?.`

`[role] ?? 0` renders a confident "0 users" against every role card. An admin reading this page concludes the workspace has no users. Separately,

`ROLE_ORDER` lists only `super_admin`, `admin`, `viewer` — `editor` is a real role used by `CMS_ROLES` route guards but is invisible on the page that documents roles.

```
GET /rest/v1/admin_users → HTTP 404 PGRST205
AdminUserAccessRoles.tsx:80    await (supabase as any).from("admin_users")
AdminUserAccessRoles.tsx:69    const ROLE_ORDER = ["super_admin", "admin",

Live: profiles = 1 row,  user_roles = 1 row
```

The `(supabase as any)` cast is how this escaped `tsc`. It appears at every one of the three dead-table call sites.

VERIFIED LIVE

RBAC

WRONG DATA SHOWN

P1 · 5

Blog Article Performance queries a table that does not exist

Third of the three dead tables. `AdminBlogPerformance` reads `article_analytics`; there is no such relation. The page has no `isError` path, so it renders as empty rather than as broken.

```
AdminBlogPerformance.tsx:66      .from("article_analytics")    → HTTP 404 PGR

Real analytics sources that do exist:
  blog_user_events                0 rows    (article_id → blog_posts.id)
  analytics_reporting_daily       261 rows
  blog_posts.view_count           counter column, populated
```

`blog_user_events` is empty too, so Reader Engagement has no data to show either — worth confirming the tracking hook is wired before rebuilding the query.

VERIFIED LIVE

BLOG

P1 · 6

About four hundred admin colour classes generate no CSS at all

The admin surface uses two syntaxes for the same tokens. The arbitrary form — `bg-[hsl(var(--admin-surface))]`, 184 usages — works. The shorthand form — `bg-admin-surface`, 85 usages — does not, because `tailwind.config.ts`'s `admin` palette has no `bg`, `text`, `surface`, `danger`, `success` or `warning` keys. Tailwind emits nothing and reports nothing. I checked the shipped stylesheet:

```
grep '\.<class>{' dist/assets/index-D9IL0spW.css      source usages  i
  text-admin-text                                196           0
  bg-admin-surface                               85            0
  text-admin-text-subtle                         55            0
  text-admin-text-muted                         37            0
  bg-admin-bg / bg-admin-surface-hover / border-...-subtle  11 ea         0
  text-admin-danger / -success / -warning, bg-...-muted    2-3 ea        0
— for comparison, these do resolve —
  border-admin-border                             —
  text-admin-primary                             —
  bg-admin-background                             —
```

Confirmed in a live browser against the running dev server. Probe spans injected into `.admin-theme` on `/admin/auth`, computed styles read back:

```
bare span (no class)      color rgb(255,255,255)  bg rgba(0,0,
text-admin-text           color rgb(255,255,255)  bg rgba(0,0,
text-admin-text-subtle    color rgb(255,255,255)  bg rgba(0,0,
bg-admin-surface          color rgb(255,255,255)  bg rgba(0,0,
bg-admin-bg               color rgb(255,255,255)  bg rgba(0,0,
— the arbitrary form, same page, same host —
```

So the elements fall back to inherited colour. The panel still reads as dark because the arbitrary-value siblings carry it — which is exactly why this has gone unnoticed. The consequence worth acting on: text intended as `--admin-text-subtle` renders at *pure white*, identical to primary body text, so the intended three-step text hierarchy collapses to one step across ~290 elements. Every "quiet" label in the admin panel is currently shouting.

Fix: add the missing keys to the `admin palette` (`bg`, `surface` / `surface-hover`, `text` / `text-muted` / `text-subtle`, `border-subtle`, `danger` / `success` / `warning` / `info` with their `-muted` variants), then pick one syntax and add a lint rule for it. A Tailwind safelist check in CI would have caught this on day one.

VERIFIED IN BROWSER

VISUAL

DESIGN SYSTEM

P1 • 7

Twenty admin tokens exist only inside `.dark`, and nine don't exist anywhere

The base `.admin-theme` block in `index.css` defines 14 tokens. Another twenty — including `--admin-background`, `--admin-primary` and `--admin-foreground` — are defined *only* in `.dark .admin-theme`, `.admin-theme.dark`. `ThemeProvider` writes the `dark` / `light` class onto `documentElement` from `localStorage["vite-ui-theme"]`, shared with the public site. If that value is ever `light`, the admin panel loses its page background, its champagne-gold brand accent, all button fills and most text colours in one step: `hsl(var(--admin-primary))` becomes `hsl()`, which is invalid, so the declaration is dropped.

```
index.css:190 .admin-theme → 14 tokens
               --admin-bg --admin-surface --admin-card --admin-border -
               --admin-grain --admin-text --admin-muted --admin-danger

index.css:254 .dark .admin-theme, .admin-theme.dark → +20 tokens defined
               --admin-background --admin-primary(+hover,-foreground,-m
               --admin-accent(+foreground) --admin-card-foreground --ad
               --admin-popover(+foreground) --admin-input --admin-ring
               --admin-secondary(+foreground) --admin-text-muted
               --admin-danger-muted --admin-info-muted --admin-success-
```

Referenced but **never defined in any block**:

```
--admin-text-subtle (9)  --admin-border-subtle (29)  --admin-error (2
--admin-wine (8) + --admin-wine-light, --admin-gold-muted,
--admin-obsidian, --admin-charcoal, --admin-stone (all mapped in tail
```

Confirmed live. I set `localStorage["vite-ui-theme"] = "light"`, reloaded `/admin/auth`, and read the tokens off the `.admin-theme` host in both states:

	root "dark lenis"	root "light lenis"
--admin-primary	43 90% 55%	(UNSET)
--admin-primary-foreground	set	(UNSET)
--admin-background	0 0% 1%	(UNSET)
--admin-foreground	0 0% 98%	(UNSET)
--admin-text-muted	215 15% 65%	(UNSET)
--admin-ring / --admin-input	set	(UNSET)
--admin-text	0 0% 100%	210 20% 98% ← falls
--admin-bg / -surface / -card / -border		unchanged ← thes
document.body background	rgb(10,10,11)	rgb(250,250,250)

The failure mode, precisely: under `light` the body ground flips to near-white while `--admin-text` stays near-white (`210 20% 98%`) and `--admin-background` stops resolving — so `AdminLayout` 's `bg-[hsl(var(--admin-background))]` is dropped and the shell shows white text on the white body. The brand gold goes with it.

One honest caveat: the login page itself barely changes visually, because it is built almost entirely from Tailwind literals (`bg-[#121212]` , `bg-amber-500` , `text-zinc-400`) rather than admin tokens — its Login button still computes to `rgb(245,158,11)` under both themes. The token loss is proven; the *visual* damage to the authenticated shell, which does consume these tokens, is still pending the Playwright run.

Fix: move the full token set into the base `.admin-theme` block and let `.dark` override only what genuinely differs; define the nine missing tokens or delete their Tailwind mappings. Cheap insurance in the meantime: have `AdminLayout` force `dark` on the admin subtree rather than inheriting a theme key it shares with the public site.

VERIFIED IN BROWSER

VISUAL

THEMING

P1 • 8

The admin landing page shows a fabricated live activity feed

`AdminHub` renders six hardcoded events from a literal array, under a heading with a working refresh button that re-fetches hub stats and never touches the list. The copy is specific enough to be believed: a named user completing a questionnaire "2 min ago", "Daily backup written to secure vault", "Invoice Dispatched", "Security Scan OK. 18 admin tokens verified. No warnings."

```
AdminHub.tsx:493-498 (rendered – not commented out)
  { time: "2 min ago", title: "Lead Quiz Complete", desc: "sharma1.aay
  { time: "12 min ago", title: "System Snapshot", desc: "Daily backu
  { time: "2 hrs ago", title: "Security Scan OK", desc: "18 admin to
...3 more
```

A fake green "Security Scan OK" on the first screen after login is the worst of these — it asserts a safety property nothing checked. Wire the panel to `audit_logs` (4 rows) or `lead_activities` (60 rows), or remove it; don't ship placeholder operational state.

VERIFIED

UX

TRUST

P1 · 9

Media Library shows two contradictory truths on one screen

`/admin/cms/media-library` renders a metrics panel computed from `media_files` above a workspace that browses `assets`. Those are different stores with different contents, so the KPI row and the grid beneath it disagree by a factor of six.

AdminMedia.tsx:52 fetchMediaFiles → .from("media_files").select("*")
AdminMedia.tsx:312 <AssetWorkspaceLayout /> → AssetService → assets + a

Live row counts: media_files **131** assets **22** asset_versions 22
 media_folders 6 asset_usages 21 asset_colle
 site_media_assets 155 media_assets 0

This is the visible face of the unfinished DAM v3 migration that ADR-0002 records. Until `media_files` is retired, derive the KPIs from whatever the workspace below actually lists, so the numbers describe what the user is looking at.

VERIFIED LIVE

DAM

UX

P1 · 10

Lint fails, so CI is red

18 errors. `lint` is a blocking step in `.github/workflows/ci.yml`. Four errors sit in the DAM files currently being edited on this branch.

```
rules-of-hooks      AssetInspector.tsx:35      useQuery called condi
Footer.tsx:23,24      hooks inside _Particles
label-has-...-control AssetSidebar.tsx:857,867
click-events-...    CollectionDetailView.tsx:416 · MegaMenu.tsx:23  (+
no-autofocus        CollectionDetailView.tsx:288
no-unused-vars      TelemetryExporterFactory.ts:1 (×2) · CollectionDetai
Checkbox.tsx:4 · Radio.tsx:4 · Badge.tsx:4 · Button.
AdminArchitecturePortal.tsx:11
```

Six of the eighteen are genuine keyboard-accessibility defects, not noise: two unlabelled form controls in the asset sidebar, and two click handlers on non-interactive elements (one in the new collection view, one in the public mega-menu) that keyboard users cannot reach.

VERIFIED

CI

A11Y

04 — MEDIUM

Correctness, observability and drift

P2 · 11

`AssetInspector` 's conditional hook — not a crash, but it leaks a subscription

I expected this to throw and it does not, so it's worth stating precisely. `AssetInspector` returns early on `!selectedAssetId`, before `useQuery`. In `AssetWorkspaceLayout` the same component type sits in branch 1 and branch 3 of one ternary at the same tree position, so React reuses the fiber across select/deselect instead of remounting. I built the equivalent tree and ran both transitions:

```
STEP0 initial      : empty          0 hooks
STEP1 select asset : asset abc-123:0 0 → 1 hooks  no throw (memoize
so React picks th
STEP2 deselect     : empty          1 → 0 hooks  no throw (the "to
only fires when a
successor; zero c
```

So the user-visible behaviour is fine today. The real cost is that on deselect React commits no effect list for the fiber, so the previous render's effect cleanup never runs and the React Query subscription is never torn down — one leaked

subscription per select/deselect cycle. It is also a blocking lint error, and it becomes a hard crash the moment a second hook is added above the early return. Move the guard below the hook and use the `enabled` flag that is already there.

CORRECTED BY TEST

DAM

REACT

P2 · 12

Recent Activity reads a table nothing writes

`RecentActivityFeed` queries `system_logs`. The table exists but holds zero rows, and no code path in `src/` or in the edge functions inserts into it. The panel is permanently empty by construction.

```
RecentActivityFeed.tsx:35  .from("system_logs")    → HTTP 200, [], 0 rows
grep -rn 'system_logs' src/ supabase/functions/    → 1 hit, read-only
```

VERIFIED LIVE

DASHBOARD

P2 · 13

The audit trail is decorative

The System module presents an Audit Logs page, implying complete coverage. `audit_logs` holds four rows, and `auditService.writeAudit` is called from eight sites. Most destructive and sensitive admin actions write nothing.

```
Audited:      AdminServices · AdminTeam · AdminTestimonials · AdminEstimate
              BlogEditorForm · BlogList · GeneralSettingsForm

Not audited:  media & asset deletes (AdminMedia, AssetService) · lead dele
              hero carousel · gallery · milestones · process steps · befor
              user create/update/delete (manage-user, invite-user) · role
              AdminUserAccessSecurity (2FA enforcement, session timeout)
              maintenance-mode toggle (AdminLayout)

AuditService.ts:27  ip_address: 'client'    ← literal string, not an a
```

The `ip_address` column is populated with the literal `'client'` on every row, so it carries no forensic value. A browser cannot see its own egress IP; capture it in an edge function or drop the column.

VERIFIED LIVE

OBSERVABILITY

COMPLIANCE

P2 · 14

Generated types have drifted from the live schema

`types_utf8.ts` describes 52 tables. The live project exposes 64. Ten live tables are missing from the types and three typed tables don't exist. Because the missing ones are reached through `(supabase as any)`, `tsc` stays green over queries that 404 at runtime — the direct cause of findings 3, 4 and 5.

```
Live but absent from types (10):
  system_logs · admin_sessions · cms_sections · crm_pipeline_history · da
  decision_events · process_faqs · process_metrics · website_events
  workspace_commitment_revisions

In code but absent from the live DB (3):
  admin_users · article_analytics · estimate_rates

Corroborated in-repo: LearningHealthDashboard.tsx:20
  "TODO: regenerate src/integrations/supabase/types_utf8.ts. It predates
```

Regenerate with `--linked` (per ADR-0003, never `--local`), then delete the `as` any casts so the compiler can see the three dead tables.

VERIFIED LIVE

SCHEMA

P2 · 15

The permission matrix contradicts the route guards

`lib/auth/permissions.ts` and `routes/adminRoutes.tsx` disagree about who may see what. Both are fail-closed on a null role, so this is not an escalation path — but any control gated on `can(...)` renders for roles that `RoleGuard` then bounces, producing dead-end navigation.

```
PERMISSIONS.cms.view      = [super_admin, admin, editor, viewer]
routes: CMS_ROLES         = [super_admin, admin, editor]           ← no

PERMISSIONS.dashboard.view = [super_admin, admin, editor, viewer]
routes: dashboard/*       = ADMIN_ONLY                            ← n

PERMISSIONS.estimate_rates = declared resource for a table that does not
```

`AdminHub`'s quick actions are not gated at all: a `viewer` sees "System Config" → `/admin/system/settings`, which is `SUPER_ONLY`. Derive one from the other, and gate the hub tiles on `can()`.

VERIFIED

RBAC

UX

P2 · 16

Estimator sidebar tab can never show as active

`EstimatorModule` links its second tab to `/admin/estimator/pricing-configuration`, which the router immediately redirects to `/admin/estimator/config`. `ModuleLayout` compares `location.pathname` against `tab.path`, so after the redirect no tab matches and the sidebar shows nothing selected on the page you're on.

```
EstimatorModule.tsx:16  { label: "Pricing Configuration", path: "/admi
adminRoutes.tsx         <Route path="pricing-configuration" element={<
```

VERIFIED

UX

NAVIGATION

P2 · 17

No list on the admin surface is paginated

`AuditService.getLogsPaginated` is the only paginated reader in the panel. Every other list fetches everything, and several fetch every column. Current volumes are small enough that nothing is slow today, which is precisely why this will land as a production incident rather than a code review comment.

```
CrmSettings.tsx:71      .from("leads").select("*")           ← 51
AdminMedia.tsx:52      .from("media_files").select("*")       ← 131
tabs/SalesTab.tsx      .from("leads").select("id, status, ...") ← no
LeadFunnelChart · LeadSourceChart · ProjectPipelineChart ← n
23 × select("*") across pages/admin, components/admin, services/
```

`leads` alone has 51 columns including seven `jsonb` blobs (`estimate_breakdown`, `discovery_*`, `alcs_evidence`). `CrmSettings` pulls all of it to the browser to build a CSV export.

VERIFIED

PERFORMANCE

P2 · 18

Five edge functions have no authorization check

Eight functions call `verifyAdmin`; the media and PostHog paths are correctly guarded. Five have neither an admin check nor a `verify_jwt = false` entry in `config.toml`, which means the platform accepts any valid JWT — and the public anon key is a valid JWT.

```
verifyAdmin present:  imagekit-upload · media-import · media-export · med
                      migrate-to-imagekit · sync-imagekit · posthog-query
                      log-lead-activity
```

No guard: `inspect-schema` · `notify-telegram` · `posthog-to-telegram` · `cda-a`

```
inspect-schema/index.ts:21-34  leftover debug endpoint
  createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY)
  adminClient.rpc('get_column_type', ...)
  adminClient.from('user_roles').insert({ user_id: '000...', role: 'INVALID
  → returns the raw Postgres error message to the caller
```

`inspect-schema` is the one to delete today: a service-role write path against `user_roles`, callable by anyone with the public key, that returns schema error text. The insert fails on the `app_role` enum so the practical impact is limited to information disclosure — but it should not exist. `notify-telegram` unguarded

means anyone can push messages into the studio's Telegram channel. I did not invoke either function to confirm deployment; check the deployed function list.

NOT PROBED

SECURITY

EDGE FUNCTIONS

P2 · 19

The Content-Security-Policy is report-only

vercel.json ships a detailed, well-scoped CSP under Content-Security-Policy-Report-Only. Browsers log violations and enforce nothing. The other headers (X-Frame-Options: DENY , nosniff) are live.

```
vercel.json  "key": "Content-Security-Policy-Report-Only"
              default-src 'self'; script-src 'self' 'unsafe-inline' ...; fra
```

If the report queue is quiet, promote it to Content-Security-Policy. Note 'unsafe-inline' in script-src will limit what it buys you.

VERIFIED

SECURITY

05 – USE CASES

Module inventory

Eight modules behind AuthGuard + AdminDeviceGate. Each module shell redirects its bare index to a first child, so there are no empty landings. Row counts are live.

MODULE	ROUTE	ROLES	JOB	PRIMARY TABLES (LIVE R
Hub	/admin	all 4	Landing, KPIs, quick actions	via useHubStats
Dashboard	/admin/dashboard/*	admin+	5 analytics tabs: overview, sales, traffic, content, system	leads 56 · projects 3 · blog_posts 9 · testimonial · analytics_reporting_dail 261
CMS	/admin/cms	editor+	12 content surfaces in 4 groups: Content, Media, Engagement, Team	projects 3 · services 8 · blog_posts 9 · testimonial · team_members 6 · gallery_items 29 · hero_me 7 · studio_milestones 6 · design_process_steps 5 · transformation_stories 3 · site_media_assets 155
CRM	/admin/crm	viewer+	Pipeline board, saved views, lead workspace,	leads 56 · lead_activities · lead_tasks 1 ·

MODULE	ROUTE	ROLES	JOB	PRIMARY TABLES (LIVE R
			analytics, CSV settings	lead_objections 0 · decision_events 12
Discovery	/admin/discovery	admin+ / super	Style-quiz analytics; archetype, adjective, material, lighting config	leads (discovery_* cols) · analytics_reporting_daily · quiz_results 0 · estimator_flow_config 5
Estimator	/admin/estimator	admin+ / super	Estimate submissions; pricing, tiers, add-ons, result templates	leads (estimator source) · estimator_flow_config 5 · estimate_rates missing
Blog analytics	/admin/blog	admin+	Overview, article performance, reader engagement	blog_posts 9 · blog_user_events 0 · article_analytics missing
User Access	/admin/user-access	admin+ / super	Admin users, role matrix, 2FA & session policy	profiles 1 · user_roles 1 · site_settings.security_con · admin_users missing
System	/admin/system	super only	Company settings, integrations, email templates, audit logs, maintenance	site_settings 1 · audit_lo · system_logs 0
Learning Health	/admin/learning-health	super only	Recommendation-engine health metrics	workspace_commitment_revis 20 · decision_events 12
Architecture Portal	/admin/architecture	admin+	Static knowledge portal, 1303 LOC, no data layer	—

What the module layer gets right

The shells are genuinely consistent: `ModuleLayout` gives every module the same grouped sidebar, portalled action slot (deliberately a portal, not context, to dodge the reference-equality re-render trap — the comment says so), and scroll container. `RoleGuard` and `can()` are both fail-closed on a null role. `AdminLayout` handles the slow-role case with a 10-second grace period and a retry that clears the `sessionStorage` role cache, rather than bouncing to login.

The global maintenance-mode banner is persistent and carries its own deactivate action.

`AdminDeviceGate` blocks below 1024px, touch-primary and mobile UAs — which is why the `hidden lg:*` sidebars are coherent rather than a responsive bug, though it also means `CrmModule`'s mobile nav Sheet is unreachable dead code.

06 – BACKEND SCHEMA

64 live tables, and six duplicate lineages

Introspected from the project's own OpenAPI document. The recurring pattern is a v1 table, a v2 replacement and a v3 replacement all still present, with reads split across them.

CONCERN	LIVE TABLE	ROWS	READ BY	VERDICT
Projects / portfolio	projects	3	SupabaseProjectRepo	CANONICAL
	portfolio	0	nothing	DROP
	project_gallery	—	project detail	DEPRECATED_IMAGE_URL
	project_categories · project_materials · project_views · daily_project_kpis	—	repo / analytics	KEEP
Blog	blog_posts	9	SupabaseBlogRepo	CANONICAL
	blogs	9	nothing	DUPLICATE CONTENT
Media / DAM	assets + asset_versions + asset_usages + asset_metadata + asset_tags + asset_tag_links + asset_collections	22 / 22 / 21 / — / — / — / 1	AssetService, DAM v3	TARGET MODEL
	media_files + media_folders	131 / 6	MediaRepository, AdminMedia KPIs	V2, STILL LIVE
	site_media_assets	155	keyed site assets	→ MEDIA_FILES FK
	media_assets	0	nothing	DROP
	studio_milestones	6	AdminMilestones	CANONICAL
	milestones	0	nothing	DROP

CONCERN	LIVE TABLE	ROWS	READ BY	VERDICT
Lead tasks	lead_tasks	1	TaskRepository	CANONICAL
	crm_tasks	0	nothing	DROP
Page content	page_sections	20	public pages	IN USE
	site_content	0	nothing	DROP
	cms_sections	—	nothing in src/	UNTYPED, UNREAD
Activity log	audit_logs	4	AuditService	BARELY WRITTEN
	system_logs	0	RecentActivityFeed	NEVER WRITTEN
Referenced, absent	estimate_rates	—	EstimatorRegistry, useEstimateEngine	404
	admin_users	—	AdminUserAccessRoles	404
	article_analytics	—	AdminBlogPerformance	404

Two god tables, and ADR-0002's unfinished half

P3 · 20 **leads** has 51 columns; **site_settings** has 43

leads has accreted four distinct concerns into one row: contact + pipeline (status, sub_status, forecast_category, assigned_to), estimator output (area, property_type, city_tier, estimated_min/max, estimate_breakdown), discovery output (archetype, confidence, sensory, lifestyle, priorities, contradictions) and ALCS output (execution_path, confidence, reasoning, evidence, primary_drivers). Two near-duplicate score columns coexist — **score** / **score_details** and **lead_score** — as do **source**, **lead_source** and **source_url**. It also explains P2·17: any `select("*")` on leads is expensive by construction.

P3 · 21 **Eight entity tables still carry raw CDN URLs**

ADR-0002 says assets are referenced through **asset_usages** and never as a raw URL on an entity row. **projects**, **services**, **blog_posts** and **project_gallery** were migrated — their old columns are renamed **deprecated_***. These were not, and their columns are not marked, so the deviation is invisible to anyone reading the schema:

```

hero_media.media_url          gallery_items.image_url
team_members.image_url        testimonials.avatar_url
transformation_stories.before_media, .after_media
milestones.icon_url           design_process_steps.image_url
awards.image_url              site_content.image_url    page_sections.

```

Two live `TODO(ADR-0002)` comments in `BlogEditorForm.tsx:149` and `PortfolioFormDialog.tsx:219` confirm the write paths still target `deprecated_cover_image_url` even on the tables that were migrated.

P3 • 22 No `current_version_id` on `assets`

`assets` holds no pointer to its live version; "current" is inferred from `asset_versions.version_number DESC`, and `AssetInspector` encodes that as a comment plus `asset_versions?.[0]`. Every consumer must now order correctly or silently show an old file. Given WS-03 just shipped non-breaking replace, an explicit `current_version_id` FK would make the invariant enforceable instead of conventional.

P3 • 23 Dead code and config drift

- `AdminMediaOld.tsx` (617 LOC) and `AdminUsers.tsx` (173 LOC) are routed nowhere and imported nowhere.
- `CrmModule`'s mobile nav Sheet is unreachable behind `AdminDeviceGate`'s 1024px floor.
- `apps/web/vercel.json` is shadowed by the root `vercel.json`, taking its `/sitemap.xml` rewrite with it.
- `scripts/admin-route-paths.js` is missing `/admin/estimator/config`, `/admin/learning-health` and `/admin/architecture`, and carries four stale entries (`/admin/access`, `/admin/cms/studio-statistics`, `/admin/system/team-members`, `/admin/system/access-control`). Its drift check only runs one direction. Low impact: the root `vercel.json` catch-all rewrite already covers hard refreshes.
- 34 hardcoded hex colours across 17 admin files, and `AdminArchitecturePortal.tsx` (1303 LOC) / `AssetSidebar.tsx` (1220 LOC) are past the point of comfortable review.

07 – RECOMMENDATION

Order of work

Sequenced so each step makes the next one safer. Steps 1–3 are hours, not days.

1 Rotate the exposed keys, then close `site_settings`

Resend, Vercel, PostHog. Then split the table behind a public view and stop `select("*")`.
Nothing else matters while credentials are being handed to visitors. P0 · 1

2 Delete `inspect-schema`, guard `notify-telegram`

A service-role write endpoint with no authorization is not worth keeping for debug convenience.
P2 · 18

3 Regenerate types with `--linked`, then remove the `as any` casts

This turns findings 3, 4 and 5 from silent runtime 404s into compile errors, and stops the next one happening. P2 · 14

4 Add `tsConfig` to dependency-cruiser and baseline the 29 violations

`arch:check` is already `continue-on-error`, so this is safe to switch on immediately and burn down after. P0 · 2

5 Fix the three dead-table screens

Estimator pricing needs the table created or the feature cut — as shipped, admin edits never reach a client quote. Roles and Article Performance need repointing at `user_roles` / `profiles` and `blog_user_events`. P1 · 3, 4, 5

6 Repair the admin token layer, then clear lint

Add the missing Tailwind `admin` keys, hoist the 20 dark-only tokens into the base block, define or delete the nine phantom tokens. Then the 18 lint errors — including six real keyboard-accessibility defects — and CI goes green. P1 · 6, 7, 10

7 Replace the fake activity feed and reconcile the media KPIs

Two small changes that stop the panel asserting things that aren't true. P1 · 8, 9

8 Then: audit coverage, pagination, duplicate-table cleanup

Extend `writeAudit` to deletes and user/role changes, paginate the lead and media lists, and drop the six confirmed-dead tables. P2 · 13, 17 · P3 · 20-23

Method. Static analysis of `apps/web/src` (routes, module shells, 36 routed pages, services, repositories, RBAC, token layers), the production build output in `dist/`, all six quality-gate commands, live schema introspection via the project's OpenAPI document, anon-key RLS probes, a live browser pass against the running dev server on the one admin surface reachable without a session, and one React reconciliation test written to check finding P2·11 — which disproved my initial reading of it.

Limits. No authenticated click-through: entering the admin password is outside what I will do, so the 30 screens behind the login were not exercised — `e2e/admin-audit-verification.spec.ts` is in the repo to close that gap on one command. Per-role RLS write policies were not inspected. Deployment state of the unguarded edge functions was not probed. A full row-count sweep and one `site_settings` column-population probe were blocked by the sandbox classifier; the exposure in P0.1 is confirmed at the table level regardless of which cells currently hold values.